Research results for repeated integer membership lookup

Task 3 · Research Methods · Software Engineering

Authors: Medetbek Kozhakhmetov and Aibar Dzhazbayev
Collection date: 5 November 2026
Repository: github.com/Medetkb/integer-membership-benchmark
Submission version: v3.0.1-task3

Objective and main finding

This study extends Task 2 by comparing repeated integer membership queries in prebuilt Python lists and sets. The primary question is whether replacing a list with a set reduces typical lookup time by more than 20% at 10,000 keys and 50% hits, while preserving correctness and keeping incremental traced allocations within 8 MiB.

The median paired speedup in the primary condition was **822.78x**, with a 95% bootstrap confidence interval of 812.66-829.02x. The median relative reduction was 99.878%. No correctness errors were observed. This finding is conditional on the tested host and workload protocol.

Experimental design

Measurements were collected on an Apple M3 computer with 8 GiB RAM, running native macOS and CPython 3.12.14. The interpreter version and study configuration match Task 2. The instrument was extended to retain actual warmup timings, phase labels and timestamps; the measured function and timing boundaries were preserved.

Parameter	Value
Container size	100, 1,000 and 10,000 unique integers
Hit rate	0%, 50% and 100%, giving nine conditions
Workload seeds	30 per condition, numbered 101-130
Timed observation	1,000 queries replayed ten times: 10,000 lookups
Warmup and evaluation	Two warmup pairs and eight evaluation pairs per seed
Method order	Balanced and shuffled; identical paired workload SHA256
Recorded evidence	270 worker processes; 1,080 warmup and 4,320 evaluation rows

Container construction was timed separately. File I/O, workload generation and allocation tracing were excluded from the lookup timer. Collection completed in 253.7 seconds. Desktop applications remained open; background load and CPU frequency were not isolated.

Data processing and statistical units

Of 5,400 JSONL rows, only the 1,080 rows labelled phase=warmup were excluded. Validation checked the expected conditions, seeds and repeats, duplicate identities, paired workload hashes and numeric fields. The IQR rule flagged 212 evaluation observations; all were retained.

The eight evaluation repeats for each method and seed were reduced to one median. Statistical summaries then used the 30 seed estimates per condition, rather than treating 240 technical repeats as independent observations. The 95% intervals for the median used 10,000 bootstrap resamples of seeds with replacement and random seed 42.

The table reports the median and interquartile range (p25-p75), in ns/query. Each observation is batch elapsed time divided by the number of lookups, including the Python loop and hit accumulation. The p95 and p99 fields in the CSV describe seed medians, not individual-query tail latency.

Keys / hit rate	List median (p25-p75), ns/query	Set median (p25-p75), ns/query	Paired speedup
100 / 0%	302.65 (302.25-303.35)	11.54 (11.52-11.56)	26.24x
100 / 50%	237.74 (236.18-239.75)	24.77 (24.58-24.96)	9.58x
100 / 100%	167.37 (165.42-168.95)	21.05 (20.90-21.11)	8.00x
1,000 / 0%	2,874.26 (2,870.62-2,879.66)	17.07 (16.95-17.19)	168.59x
1,000 / 50%	2,166.77 (2,154.86-2,179.83)	25.27 (25.14-25.60)	85.77x
1,000 / 100%	1,465.21 (1,436.06-1,476.12)	21.28 (21.08-21.45)	68.44x
10,000 / 0%	31,708.97 (31,614.36-31,810.41)	12.87 (12.79-13.02)	2456.65x
10,000 / 50%	23,784.66 (23,582.64-24,127.62)	28.99 (28.81-29.52)	822.78x
10,000 / 100%	15,768.29 (15,633.58-15,982.92)	24.47 (24.04-24.71)	642.62x

Primary hypothesis test

H0 states that the population median relative reduction is at most 20%; H1 states that it exceeds 20%. All 30 primary-condition seed estimates were strictly above the threshold. The one-sided exact sign test gave $\mathbf{p = 9.31 \times 10^{-10}}$ at $\alpha = 0.05$. H0 is rejected under the conditional independence assumption for seeded workloads. Only the 10,000-key, 50%-hit condition is confirmatory; the other conditions are exploratory.

For each seed, speedup is its list estimate divided by its set estimate. The reported result is the median of these paired ratios and may differ from the ratio of two separately calculated medians. Ties at the 20% threshold count as failures. Practical support also requires all correctness and memory guardrails to pass.

Scientific visualization

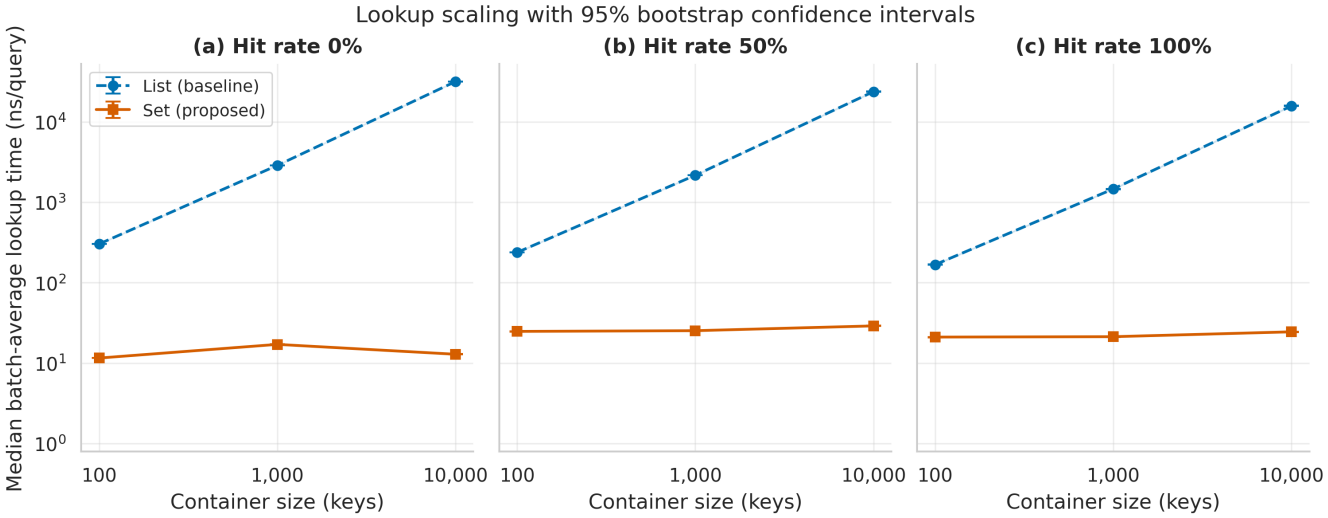


Figure 1. Median of 30 seed estimates versus container size, separately for three hit rates. Both axes are logarithmic. Error bars and shaded bands show 95% bootstrap intervals based on 10,000 resamples of the 30 seeds; some intervals are narrower than their markers. Blue dashed circles identify list; orange solid squares identify set. List lookup time increases strongly with key count.

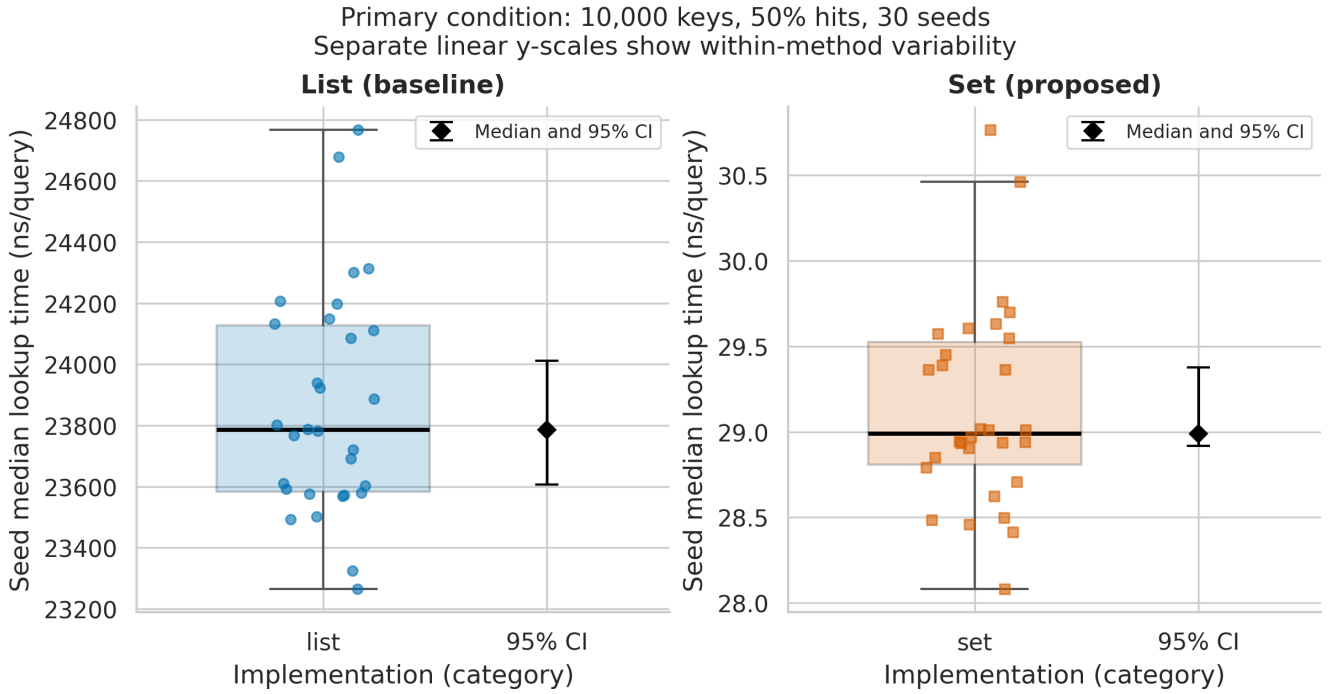
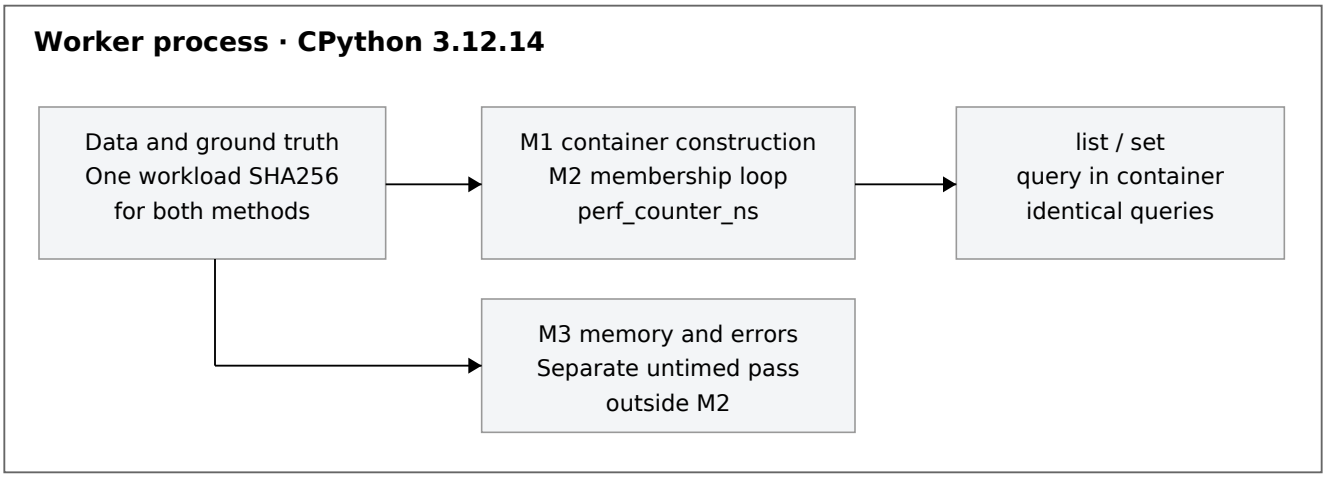


Figure 2. Primary condition: 10,000 keys and 50% hits. All 30 seed medians are shown. Boxes span p25-p75, the central line is the median, and whiskers reach observations within 1.5 IQR. The black diamond and caps show the median and its 95% interval. The panels use **different linear y-scales** to reveal within-method variation; compare absolute magnitudes using Figure 1 and the table.

Both figures are supplied separately as vector PDF and 300 DPI PNG. A colorblind-safe palette is supplemented by distinct markers and line styles so that the methods remain identifiable in monochrome.

Domain model and measurement points

The selected track is Software Engineering. The full Mermaid C4 component diagram is in docs/domain_model.md. It expands one worker process and identifies the controller, workload generator, probes, list/set implementations, observation buffer and raw-log flow. The compact diagram below highlights the measurement boundaries.



Buffer → controller → JSONL → analysis and figures; file I/O outside M2

Metric	Definition and scope
Lookup time	L = T / (Q × B), with T in ns/batch, Q=1,000 and B=10.
Throughput	X = 10^9 / L, queries/s; derived from the same timer.
Seed estimate	Median of eight repeats; overall estimate is the median of 30 seed estimates.
Speedup and reduction	S = L_list / L_set; R = 1 - L_set / L_list, within seed.
Errors	Fraction of observations with errors > 0; not a per-query error probability.
Memory	tracemalloc peak for the container and Boolean vector; not process RSS.

Memory and element-wise correctness are checked before warmup, once per method and worker, and their values are copied to its observation rows. Therefore, warmup is not an untouched process cold start, and repeated memory values are not independent measurements.

Guardrails

Failed observations: **0 of 4,320** evaluation rows and **0 of 1,080** warmup rows. The maximum traced allocation peak across all conditions was **0.6253 MiB**, against an 8 MiB limit. In the primary condition, peaks were 0.0848 MiB for list and 0.6253 MiB for set. Zero observed failures does not prove zero error probability for other inputs.

Discussion and limitations

In the primary condition, median lookup time was 23,784.66 ns/query for list and 28.99 ns/query for set. Median construction time was 101.96 microseconds for set versus 14.89 microseconds for list. Faster repeated lookup does not imply lower total cost for a single query when container construction is included.

This is one experiment on one host using synthetic integers. Seeds vary workloads, not physical machines. The intervals describe workload variation within this run; they do not capture between-machine or between-session uncertainty. Open desktop applications and uncontrolled CPU frequency remain sources of noise. Balanced randomized order reduces systematic ordering effects but cannot remove environmental variation.

Uniform sampling and the absence of deliberately adversarial hash collisions limit external validity. The study does not establish a universal Python speedup, and its findings cannot be applied directly to strings, databases, network services or one-off queries. The proposed implementation uses the built-in set; it is not a new algorithm.

Reproducibility and submission artifacts

Raw JSONL and provenance are retained under data/raw/ and data/provenance/. Processing is modularized in src/process_data.py and visualization in src/visualize.py. The executed notebooks/01_data_analysis.ipynb contains eight sequential code cells with saved tables and figures. Bootstrap seed is 42. CSV summaries include means, sample SD, medians, p25, p75, p95, p99 and 95% intervals.

To reproduce the analysis, install requirements-analysis.txt and run **python src/run_notebook.py** from the repository root. This starts a fresh kernel without recollecting timings. The README provides a separate command for new measurements in a new output directory. Existing collection evidence is protected against overwriting.

Seventeen automated checks passed for instrument correctness, timeouts, preservation of previous runs, phase filtering, retention of errors and outliers, complete paired data, and deterministic bootstrap calculations. Figures and this PDF were visually inspected. Runtime environments, notebook checkpoints and additional raw runs are excluded from Git. Submit the public repository version link and this English report in LMS.

Technical references

Python 3.12 - perf_counter_ns

NumPy - Generator.choice

C4 model - Component diagram

NBClient - Executing notebooks

Code, analysis and document preparation were assisted by Codex. No additional individual contributions are attributed beyond the existing CONTRIBUTIONS.md record. Both authors should review and understand the methods and findings before submission.